

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 25

Duration :60 Mins
On Class
Total Marks:50

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I _____Prartana T(192421099)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Prartana T

Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:

id + id

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aS \mid b$

Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:

$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.

$S \rightarrow Sa \mid b$

Justify your answer.

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Q7. Construct a parse tree for the string:

a + b

using the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:

id + id

Q11. Identify the handle in the string:

id + id

for the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q12. Construct the operator precedence relations for the operators:

+
*

where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
 $\text{id} * \text{id}$

Q14. What is an LR parser? Illustrate the parsing process using the grammar:
 $S \rightarrow aA$
 $A \rightarrow b$

Q15. Given the grammar:
 $S \rightarrow aA$
 $A \rightarrow b$
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
 $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Q17. Construct the augmented grammar for:
 $S \rightarrow a$

Q18. Find the closure of the LR(0) item:
 $S' \rightarrow \bullet S$
for the grammar:
 $S \rightarrow a$

Q19. Check whether the grammar is ambiguous:
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$
using the string:
 $\text{id} + \text{id} + \text{id}$

Q20. Construct a leftmost derivation for the string:
 $\text{id} + \text{id}$
using the grammar:
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$

Q21. Apply left factoring to the grammar:
 $S \rightarrow aA$
 $S \rightarrow aB$
 $A \rightarrow b$
 $B \rightarrow c$

Q22. Find FIRST(A) for the grammar:
 $A \rightarrow x$
 $A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.
 $S \rightarrow aA \mid aB$
 $A \rightarrow b$
 $B \rightarrow c$

Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Show the table entries for the terminals:

a, b, \$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:
id + id

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aS \mid b$

Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:

$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.

$S \rightarrow Sa \mid b$

Justify your answer.

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Q7. Construct a parse tree for the string:

a + b

using the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:

id + id

Q11. Identify the handle in the string:

id + id

for the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q12. Construct the operator precedence relations for the operators:

+

*

where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:

id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:

$S \rightarrow aA$
 $A \rightarrow b$

Q15. Given the grammar:

$S \rightarrow aA$
 $A \rightarrow b$

Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Q17. Construct the augmented grammar for:

$S \rightarrow a$

Q18. Find the closure of the LR(0) item:

$S' \rightarrow \cdot S$
for the grammar:
 $S \rightarrow a$

Q19. Check whether the grammar is ambiguous:

$E \rightarrow E + E$
 $E \rightarrow id$
using the string:
 $id + id + id$

Q20. Construct a leftmost derivation for the string:

$id + id$
using the grammar:
 $E \rightarrow E + E$
 $E \rightarrow id$

Q21. Apply left factoring to the grammar:

$S \rightarrow aA$
 $S \rightarrow aB$
 $A \rightarrow b$
 $B \rightarrow c$

Q22. Find FIRST(A) for the grammar:

$A \rightarrow x$
 $A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.

$S \rightarrow aA \mid aB$
 $A \rightarrow b$
 $B \rightarrow c$

Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:

$S \rightarrow aS$
 $S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:

$S \rightarrow aS$
 $S \rightarrow b$

Show the table entries for the terminals:

$a, b, \$$

- 1) Given
 token stream:- id + id
 Grammar: $E \rightarrow E + E$
 $E \rightarrow id$

Sol
 $\rightarrow$ input - tokens
 id + id

$\rightarrow$ parser from E
 E

$\rightarrow$ production

$E \rightarrow E + E \rightarrow$

```

      E
     / \
    E + E
  
```

$\rightarrow$ Replace left E $\rightarrow$ Replace right E $\Rightarrow$

```

      E
     / \
    id + id
  
```

$E \rightarrow id$ $E \rightarrow id$

$\therefore$ Token sequence $\rightarrow id + id$ syntactically valid.

- 2) Given $S \rightarrow aS | b$

$S \rightarrow aS$ or $S \rightarrow b$

The grammar generated is

b
 aab
 aaab
 ...

$\therefore$ The string aaab is possible

- 3) Given

$A \rightarrow Aa | b$

To remove left recursion

$A \rightarrow Aa | b$

$A \rightarrow A\alpha | B$

$A \rightarrow BA'$

$A' \rightarrow \alpha A' | \epsilon$

$A \rightarrow bA'$

$A' \rightarrow aA' | \epsilon$

4) Given
 $S \rightarrow aA \mid b$
 $A \rightarrow c \mid \epsilon$

$FIRST(S) = (a, b)$
 $= (a, b)$

5) Given
 $S \rightarrow Sa \mid b \rightarrow$ non terminal - left side again as S 1st symbol on right
 $S \rightarrow b$

$S \rightarrow Sa$
 $\rightarrow Saa$
 $\rightarrow Saaa$
 $\dots$

$\therefore$ not suitable for topdown parsing as it contains left recursion.

4) Grammar
 $S \rightarrow aA$
 $A \rightarrow b$

LR Parser.

Step	Stack	Input	Action
1.	$\$a$	$b\$$	Shift
2.	$\$ab$	$\$$	Shift
3.	$\$aA$	$\$$	Reduce $A \rightarrow b$ $S \rightarrow ab$
4.	$\$S$	$\$$	Reduce $S \rightarrow aA$
5.	$\$ \epsilon$	$\$$	Accept

LR parsing - bottom-up parsing method, uses shift and reduce action to recognise valid string.

15) $S \rightarrow aA$, $A \rightarrow b$, SLR parsing table

Grammar:

(1) $S \rightarrow aA$

(2) $A \rightarrow b$

Augment $S' \rightarrow S$

1. $S \rightarrow aA$

2. $A \rightarrow b$

LR(0) item sets

$S \rightarrow \cdot S \rightarrow S' \rightarrow \cdot S$ (I1)

$S \rightarrow \cdot aA \rightarrow S \rightarrow a \cdot A \rightarrow S \rightarrow aA \cdot$ (I3)

$\rightarrow A \rightarrow \cdot b$ (I2)

$\rightarrow A \rightarrow b \cdot$ (I4)

16) FOLLOW(A)

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

FOLLOW(A):

$S \rightarrow AB$ (A is immediately followed by B)

FIRST(B): $B \rightarrow b$

$= \{b\}$ (Terminal b can appear immediately after A)

$\therefore \text{FOLLOW}(A) = \{b\}$

17) Augmented Grammar:-

$S \rightarrow a$

LR parsing $\rightarrow$ new S'

$S' \rightarrow S$

$S \rightarrow a$

$\therefore$ Answer:- $S' \rightarrow S$

$S \rightarrow a$

18) LR(0)

Given $S' \rightarrow \cdot S$

grammar:- $S \rightarrow a$

$$S \rightarrow a$$

no further non-terminals immediately after dot, closure is complete

$$\text{closure } \{ S' \rightarrow \cdot S \} = \{ S' \rightarrow \cdot S, S \rightarrow \cdot a \}$$

2 LR(0) items

19) Ambiguous Grammar

$$E \rightarrow E + E$$

$$E \rightarrow id$$

String

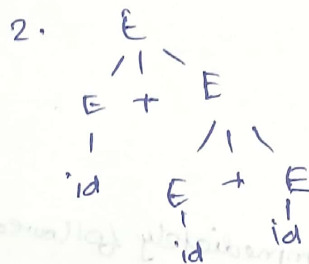
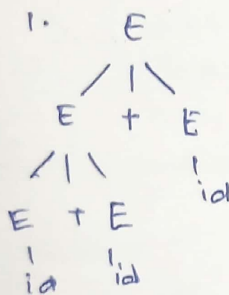
$id + id + id$

Possibility 1:

$(id + id) + id$

possibility 2

$id + (id + id)$



20) Given

$$E \rightarrow E + E$$

$$E \rightarrow id$$

String

$id + id$

(Replace leftmost nonterminal first)

$$E \rightarrow E + E$$

$$\rightarrow id + E$$

$$\rightarrow id + id$$

21) Left factorize grammar,

$$S \rightarrow aA$$

$$S \rightarrow aB$$

$$A \rightarrow b$$

$$B \rightarrow c$$

Common prefix = a

$$a \rightarrow \alpha \quad A, B \rightarrow \beta$$

$$S \rightarrow aS'$$

$$S' \rightarrow A | B$$



22) FIRST (A)

Given $A \rightarrow x$
 $A \rightarrow y$

production starts with x & y

$\therefore \text{FIRST}(A) = \{x, y\}$

23) Predictive Parsing.

$S \rightarrow aA|aB$

$A \rightarrow b$

$B \rightarrow c$

$\text{FIRST}(S \rightarrow aA) = \{a\}$

$\text{FIRST}(S \rightarrow aB) = \{a\}$

left factorizing

$S \rightarrow aS'$

$S' \rightarrow A|B$

24) Recursive descent parser.

Grammar:-

$S \rightarrow aS$

$S \rightarrow b$

$S()$

{

if (next token == 'a')

{

match('a');

$S()$;

}

else if (next token == 'b')

{

match('b');

}

else

error();

}

25) Grammar

$$S \rightarrow aS$$

$$S \rightarrow b$$

terminals : $a, b, \$$

$$\text{FIRST}(aS) = \{a\}$$

$$\text{FIRST}(b) = \{b\}$$

Predictive Parsing table.

Non terminal	a	b	\$
S	$S \rightarrow aS$	$S \rightarrow b$	

ip: $a \rightarrow$ use $S \rightarrow aS$

$b \rightarrow$ use $S \rightarrow b$

$\$ \rightarrow$ error.

6) Given

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then else } S.$

sol if E then S common prefix.

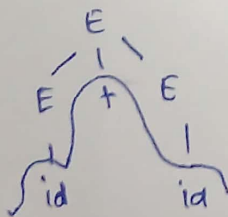
Factor common prefix into new non terminal

$S \rightarrow \text{if } E \text{ then } S S'$

$S' \rightarrow \text{else } S / \epsilon$

7) Given
 $a + b$

Grammar
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$



$\therefore$ The parse tree represents
 $a + b,$

8) Each non terminal get a procedure,

voids()

```
{
  match('a');
  A();
}
```

```
}
void A()
{
  match('b')
}
```

}

9) Given

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

$FIRST(A) = \{a\}$

$FIRST(B) = \{b\}$

$FIRST(S) = \{a\}$

non terminals	a	b	\$
S	$S \rightarrow AB$		
A	$A \rightarrow a$		
B		$B \rightarrow b$	

No production applicable -

10) input : id + id

Grammar :- $E \rightarrow E + E$

$E \rightarrow id$

Step	input	Action
\$	id + id \$	Shift id
\$id	+ id \$	Reduce $id \rightarrow E$
\$E	+ id \$	Shift +

11) Given

$$E \rightarrow E + E$$

$$E \rightarrow id$$

ip: $id + id$

↑
Handle

After reducing : $E + id$

Then $E + E$

∴ initial handle is id

12) Operators

$+$, $*$

Given

$*$ has higher precedence than $+$

∴ $*$ $>$ $+$

and

$+$ $<$ $*$

Precedence Table

operator	$+$	$*$
$+$		$<$
$*$	$>$	

13) Expression

$id * id$

The parser first recognize id and reduce it to E

Then it encounters : $*$

and shifts the operator,

$id * id$

↓

$E * E$

↓

E

∴ $E \rightarrow E * E$